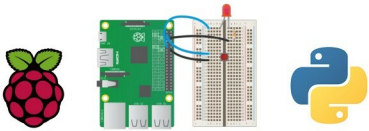
An Introduction to Raspberry Pi GPIO Programming Using Python

This article uses the RPi.GPIO Python package to introduce Raspberry Pi GPIO programming. The GPIO package bundled with Raspbian is aimed at Raspberry Pi beginners who are familiar with Python and interested in designing IoT products.



Raspberry Pi is a sensational single-board computer (SBC) and development board, which is heavily used for the prototyping of IoT based products. It is the best-selling British computer ever, with more than 10 million pieces sold already. Contrary to common belief, Raspberry Pi is not entirely open source. Yet, it has been used in many open source projects. In this article, we will learn to use it as a development board. For all the programs and circuits discussed here, I have used Raspberry Pi 3. Unless explicitly specified otherwise, from now onwards, the name Pi refers to Raspberry Pi 2 Model B, 3 Model B or Zero.

Raspberry Pi pinout

What makes Raspberry Pi suitable for making IoT projects is its 40-pin expansion header. Pins are arranged in a 2x20 fashion. Figure 1 is a pinout diagram, taken from Wikipedia. Please visit *https://pinout.xyz* for a detailed live pinout diagram.

Numbering systems

Raspberry Pi pins are numbered in two different ways—physical numbering and Broadcom numbering (BCM). In the first case, the pins are numbered sequentially from one to 40. In the figure, this is shown as *Pin#*. And the image represents what is seen when the Pi is held with the USB ports facing downwards. That is, Pin 2 is the pin in the corner. The Broadcom numbering system is the default option for the SoC (System-on-Chip). Raspberry Pi uses a SoC developed by Broadcom Limited. Raspberry Pi 2 and Zero use BCM2836 and BCM2835, while the Pi 2 version 1.2 and 3 use BCM2837. This is also known as GPIO numbering and is shown as *GPIO#* in the figure.

As you have probably guessed already, all the pins are not programmable. There are eight ground pins and two +5V pins and three +3.3V pins, which are not programmable. There are other dedicated pins too. Most of the pins have alternative functions, as shown in the figure. A majority of the pins are directly connected to the SoC; so while connecting circuits or components, one should be careful to avoid wrong wiring and short circuits. It is always good to have a descriptive pinout diagram printed out for quick reference as well as a multimeter on the work desk.

Programming the pins

Armed with some understanding about the pins, let us move to programming. The Python package used for Raspberry Pi GPIO programming is *RPi.GPIO*. It is already installed in Raspbian, the default operating system for Pi.

If you are using any other operating system, the package can be installed by using the following command:

```
$ sudo pip install RPi.GPIO
```

Now, fire up IDLE or the Python console and type the following commands:

```
import RPi.GPIO as GPIO
print(GPIO.RPI_INFO)
```

You will get some basic information about your Pi printed as the output. So the 'Hello world' of GPIO programming is done!

The GPIO programming workflow

Let us discuss the steps involved in writing a GPIO Python script, in general.

Step 1: Import the *RPi.GPIO* package.

Step 2: Set the numbering style to be used. We use the method *GPIO.setmode()* for this. It takes either *GPIO.BOARD* or *GPIO.BCM* as the parameter. *GPIO.BOARD* stands for physical numbering and *GPIO.BCM* stands for Broadcom numbering. In order to keep things simple, we will